Secure URL Handling and Validation Plan for PhishGuard

Prepared by

Luis Nicaragua
Research and Documentation
PhishGuard Team

Purpose of This Document

The purpose of this document is to support PhishGuard's next prototype development cycle by focusing specifically on how the application should handle user-submitted URLs. This document is different from the previous secure development research because it does not only discuss general secure software practices. Instead, it focuses on practical URL handling decisions that can directly support the PhishGuard prototype.

PhishGuard allows users to enter suspicious URLs for analysis. Because of that, the prototype must treat every submitted URL as untrusted input. The system should validate the URL, avoid automatically opening it, display it safely, and explain any detected warning signs clearly to the user.

This document is meant to help the team turn research into prototype planning, acceptance criteria, and QA testing ideas.

Research Focus

The main research question for this document is:

How should PhishGuard safely validate, handle, display, and explain user-submitted URLs in the prototype?

This research supports the following prototype areas:

- URL input
- URL validation
- Safe URL display
- Scan result behavior
- Sonar Analysis Log warnings
- Bait Breakdown explanations
- Recommended user actions
- Future optional URL threat-checking

Why This Research Matters for PhishGuard

PhishGuard is designed to help users make safer decisions when they encounter suspicious emails, text messages, or URLs. However, because the prototype works with suspicious URLs, the application itself must avoid unsafe behavior.

For example, PhishGuard should not automatically open a submitted link. It should not make a suspicious URL appear safe before analysis. It should not display the link in a way that encourages the user to click it. If the user enters invalid input, the system should provide a clear error message.

This research helps the team plan safer prototype behavior before implementation.

Source 1: OWASP Input Validation Cheat Sheet

The OWASP Input Validation Cheat Sheet supports the idea that user input should be checked before it is processed. For PhishGuard, this is important because a URL entered by a user should not be trusted automatically.

This source supports using validation rules before accepting or analyzing a submitted URL. PhishGuard should check whether the input follows a valid URL format and whether it uses an expected protocol such as HTTPS or HTTP. If the input is empty, incomplete, or not formatted like a URL, the prototype should show a clear error message.

How This Applies to PhishGuard

This source supports the URL input and validation part of the prototype.

Prototype planning decisions:

- The system should validate the submitted URL before scan results are shown.
- The system should reject empty or invalid input.
- The system should treat user-submitted URLs as untrusted input.
- The system should show clear error messages when validation fails.
- The system should avoid processing unexpected input as if it were safe.

Source 2: OWASP Server-Side Request Forgery Prevention Cheat Sheet

The OWASP Server-Side Request Forgery Prevention Cheat Sheet supports this research because it explains the risk of applications making requests to user-supplied URLs. This is important for PhishGuard if the prototype ever checks a submitted URL from the server side.

If PhishGuard only analyzes URL text locally or through simple rule-based checks, the SSRF risk is lower. However, if the application later fetches the URL, previews the page, follows

redirects, or checks the website from a server, the team must be careful. A user-submitted URL could point to an internal system, private address, or unexpected destination.

How This Applies to PhishGuard

This source supports future planning for safe URL checking.

Prototype planning decisions:

- The current prototype should avoid automatically fetching or opening submitted URLs.
- If server-side URL checking is added later, the team should restrict what destinations can be requested.
- The system should avoid blindly following redirects from user-submitted URLs.
- The team should document whether URL analysis is rule-based, client-side, or server-side.
- The team should treat any future URL preview or fetch feature as a higher-risk feature.

Source 3: MDN URL API Documentation

The MDN URL API documentation supports practical JavaScript development for URL parsing. Since PhishGuard may use a React frontend or JavaScript-based prototype behavior, the URL API can help the team validate and break down a submitted URL into parts.

The URL constructor can help identify whether a submitted value is a valid URL. If the URL is invalid, JavaScript can throw an error, which the prototype can catch and turn into a user-friendly error message.

How This Applies to PhishGuard

This source supports basic prototype implementation planning.

Prototype planning decisions:

- The prototype can use URL parsing to check whether input is valid.
- The system can extract parts of the URL, such as protocol, hostname, pathname, and search parameters.
- The system can check whether the protocol is HTTP or HTTPS.
- The system can identify whether HTTPS is missing.
- The system can produce clear error messages when the input cannot be parsed.

Example behavior:

If the user enters:
example

The system should respond:
Please enter a valid URL, such as https://example.com.

If the user enters:
http://example.com

The system can identify:
Protocol is HTTP, not HTTPS.

Source 4: Google Safe Browsing API Documentation

Google Safe Browsing supports a possible future direction for PhishGuard. This source explains that client applications can check URLs against lists of unsafe web resources, including phishing and malware sites.

This does not need to be part of the current prototype. PhishGuard can still remain realistic for Capstone 1 by focusing on rule-based checks and clear explanations. However, Google Safe Browsing can be documented as a possible future enhancement if the team wants to explore more advanced URL threat-checking later.

How This Applies to PhishGuard

This source supports future feature planning, not required current implementation.

Prototype planning decisions:

- Google Safe Browsing can be listed as a future optional enhancement.
- The current sprint does not need to rely on an external API.
- The team can continue using rule-based indicators for the prototype.
- If added later, API usage should be planned carefully with privacy, permissions, and implementation limits in mind.

This source helps the team distinguish between current scope and future growth.

Updated Development Direction for PhishGuard

Based on this research, the next development direction should focus on making URL handling safer and clearer.

The prototype should:

- Accept a URL from the user.
- Validate the URL format.

- Avoid automatically opening the submitted URL.
- Display suspicious URLs carefully.
- Show a risk result based on visible indicators.
- List scan warnings in the Sonar Analysis Log.
- Use Bait Breakdown to explain warnings in plain language.
- Provide a recommended user action.
- Show clear error messages when input is invalid.
- Treat advanced threat-checking APIs as future enhancements.

Recommended URL Handling Flow

The following flow can guide future prototype planning:

1. User enters a URL.
2. PhishGuard checks whether the input is empty.
3. PhishGuard checks whether the input is formatted as a valid URL.
4. PhishGuard identifies basic URL parts, such as protocol and domain.
5. PhishGuard checks for warning indicators.
6. PhishGuard displays a risk result.
7. PhishGuard shows Sonar Analysis Log warnings.
8. Bait Breakdown explains what each warning means.
9. PhishGuard gives a recommended user action.
10. The system avoids automatically opening the suspicious URL.

Warning Indicators to Support

The research supports planning around the following indicators:

- Missing HTTPS
- Invalid URL format
- Suspicious keywords
- URL shorteners
- Long or complex URLs
- Suspicious redirects
- Unknown or unusual domain patterns
- Requests for sensitive information
- Possible impersonation attempts

These indicators should be connected to scan warnings, Bait Breakdown explanations, and recommended actions.

Development Planning Notes

URL Input

The URL input area should be simple and clear. The user should know where to paste the suspicious URL.

Planning notes:

- Input should accept a URL as text.
- The system should check for empty input.
- The system should check for valid URL format.
- The system should not open the URL automatically.

URL Validation

URL validation should happen before the system displays a scan result.

Planning notes:

- Check whether the input can be parsed as a URL.
- Check whether the protocol is HTTP or HTTPS.
- Flag missing HTTPS as a warning.
- Return a clear error message if validation fails.

Safe URL Display

The submitted URL should be displayed carefully.

Planning notes:

- Avoid making suspicious URLs automatically clickable.
- Show the submitted URL as plain text when possible.
- Do not label a URL as safe unless the system has enough reason to do so.
- Make the risk result and warning labels more visible than the link itself.

Sonar Analysis Log

The Sonar Analysis Log should list detected warnings.

Planning notes:

- Use consistent warning names.
- Keep warning labels short.
- Connect each warning to a risk explanation.
- Prepare each warning for a Bait Breakdown explanation.

Bait Breakdown

Bait Breakdown should explain the meaning of each warning.

Planning notes:

- Explain what was detected.
- Explain why it may be risky.
- Give a recommended user action.
- Use plain language.
- Keep the explanation short enough for the interface.

Recommended User Actions

Recommended actions should help users respond safely.

Planning notes:

- Do not click suspicious links.
- Do not enter passwords or personal information.
- Verify the sender or destination.
- Visit the official website directly.
- Treat suspicious redirects and shortened links with caution.

Suggested Acceptance Criteria

- The system should allow users to enter a URL for review.
- The system should detect when the URL input is empty.
- The system should detect when the URL format is invalid.
- The system should show a clear error message for invalid input.
- The system should not automatically open the submitted URL.
- The system should display the submitted URL safely.
- The system should identify whether HTTPS is missing.
- The system should display scan warnings in the Sonar Analysis Log.
- The system should connect warnings to Bait Breakdown explanations.
- The system should provide at least one recommended user action.
- The system should keep Google Safe Browsing or similar API checks as a future optional enhancement unless the team confirms it is in scope.

Suggested QA Test Cases

Test Case 1: Empty Input

Input:
Blank field

Expected Result:
The system shows an error message asking the user to enter a URL.

Test Case 2: Invalid URL Format

Input:
hello world

Expected Result:
The system shows an error message explaining that the input is not a valid URL.

Test Case 3: HTTP URL

Input:
http://example.com

Expected Result:
The system accepts the URL but flags missing HTTPS as a warning.

Test Case 4: HTTPS URL

Input:
https://example.com

Expected Result:
The system accepts the URL and continues to the scan result.

Test Case 5: Shortened URL

Input:
A URL using a known shortener format

Expected Result:
The system flags the URL shortener and provides a Bait Breakdown explanation.

Test Case 6: Suspicious Redirect

Input:
A URL that appears to include redirect behavior

Expected Result:
The system flags suspicious redirect behavior and recommends verifying the destination.

Test Case 7: Safe Display

Input:
Any suspicious URL

Expected Result:
The system displays the submitted URL without automatically opening it.

How This Document Advances the Project

This document advances the project because it moves beyond general research. It gives the team practical guidance for how PhishGuard should handle URLs in the prototype.

The previous research explained why phishing awareness and explainable warnings matter. This document explains how the prototype should behave when a user submits a suspicious URL.

This makes the research more useful for:

- Sprint planning
- Prototype development
- QA test planning
- Acceptance criteria
- Feature documentation
- Future project artifacts

Conclusion

The next stage of PhishGuard development should focus on secure URL handling and validation. Since PhishGuard is built around suspicious URL analysis, the prototype must carefully handle user-submitted links.

The research suggests that PhishGuard should validate input, avoid automatically opening suspicious URLs, display links safely, explain scan warnings clearly, and provide recommended actions. This keeps the project realistic for Capstone 1 while still supporting cybersecurity-focused development practices.

This document can help the team turn research into practical development planning for the next sprint.

Works Cited

Google. “Google Safe Browsing.” Google for Developers, developers.google.com/safe-browsing.

MDN Web Docs. “URL: URL() Constructor.” Mozilla, developer.mozilla.org/en-US/docs/Web/API/URL/URL.

OWASP Foundation. “Input Validation Cheat Sheet.” OWASP Cheat Sheet Series, cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html.

OWASP Foundation. “Server Side Request Forgery Prevention Cheat Sheet.” OWASP Cheat Sheet Series, cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html.